

Taima Watfeh

Event Booking · Weekly Mini Project 02 · Team 4

Frontend

Responsibility: Events Screen and Filters (Flow A)

You own the first screen every visitor sees. It must show all upcoming events clearly, let people filter them, and always show the latest numbers. You implement Flow A (Discover events) from the specification. The project must be finished in two days, so build the core requirements first and touch any bonus only after everything else works.

You provide

- The Events page and its filters
- Links from every event card to Event Details (to Badr)

You depend on

- Allaith: the small skeleton (you start without it, using your own folder and mocks)
- Majed: GET /events and GET /events/categories (you start with mocks)

Working rule: all business rules are enforced in the backend. If something is unclear or the contract needs a change, tell the team leader (Allaith) so every file stays consistent.

Your Flow, Phase by Phase

Phase 1: Start Immediately with Mocks

Understand the flow and build on mock data, without waiting for anyone.

- ☐ Read API_CONTRACT.md fully. Build only to the shapes in it.
- ☐ Allaith pushes a small skeleton in his first hours (router, empty pages, types, StatusBadge, ErrorMessage, Loading). Do not wait: start on your own screen in your own folder with mock data, and use the shared pieces as soon as they are pushed.
- ☐ Keep your mock data in your own file (for example src/mocks/), exactly matching the types in the contract.
- ☐ Own your api file (listed in your responsibilities) with two versions of each function, one mock and one real. A single switch in .env (VITE_USE MOCK=true) selects which is used.
- ☐ Read sections 6, 8 and 9 (Flow A) of the specification.
- ☐ Your api file: src/api/eventsApi.ts (events list and categories). Plan components: EventsPage, EventCard, FiltersBar, EmptyState.
- ☐ Mock at least 8 events covering all four statuses (including Full and Past) and several categories, so you can test every filter.

Phase 2: Events List

Show all upcoming events.

- ☐ Fetch events from GET /events using the API client when the page opens.
- ☐ Each card shows: title, category, date, time, location, remaining places and the status badge.
- ☐ Remaining text: "N places left", or "Full" with 0 remaining when full.
- ☐ Format dates and times in a readable way (for example "Sat, 14 Nov · 18:00").
- ☐ Show a loading indicator while fetching and a clear error message with a Retry button if the request fails. Never leave a blank screen.

Phase 3: Filters

Let the user narrow the list, with filters that combine.

- ☐ Category: a dropdown filled from GET /events/categories, with an "All categories" option.
- ☐ Date: a date input.
- ☐ "Only events with available places": a checkbox.
- ☐ Send all active filters together as query parameters; the backend does the filtering. Do not filter in the frontend.
- ☐ A "Clear filters" button resets all three and shows all upcoming events again.
- ☐ When nothing matches, show a clear message such as "No events match your filters" with a button to clear them.

Phase 4: Past Events

Handle past events as the specification describes.

- ☐ Past events are hidden by default.
- ☐ Add a "Show past events" toggle that requests `includePast=true` and displays them in a separate section titled "Past events".
- ☐ Past cards are visibly marked (Past badge, muted look) and offer no booking action.

Phase 5: Navigation and Fresh Data

Keep every number correct after other screens change data.

- ☐ Clicking a card (or a View details button) opens `/events/:id`, Badr's screen.
- ☐ Reload the list every time the page is opened, so after a booking or a cancellation the remaining places and statuses are already updated. Do not cache old results.
- ☐ Test: book on another screen, come back, and confirm the card shows the new remaining and status. Test a page refresh as well.

Phase 6: Switch from Mock to the Real Backend

When the backend owner announces an endpoint is ready, connect your screen to it.

- ☐ Set `VITE_USE MOCK=false` (or switch just your own api function) and run your screen against the real backend.
- ☐ Fix any difference between your mock and the real response, then tell the owner if the backend deviates from `API_CONTRACT.md`.
- ☐ Re-test every state on the screen with real data: success, loading, error and empty.

Phase 7: Polish, Testing and Presentation

Finish and prepare.

- ☐ Check the screen on a narrow window; the layout must stay readable.
- ☐ Walk through Flow A completely: open, filter by category and date, filter by availability, clear, all events return.
- ☐ Make sure Full and Almost full events are clearly distinguishable by badge.
- ☐ Presentation topic: product walkthrough of the Events screen and the filters. Prepare it in English and practice it.

Definition of Done

- ☐ Events show all required fields and the latest remaining places
- ☐ Category, date and availability filters work alone and together
- ☐ Clear filters restores all upcoming events; empty result has a clear message
- ☐ Past events hidden by default and clearly marked when shown
- ☐ Loading and error states exist; no blank screen

Your Part in the Final English Presentation

Product walkthrough of the Events screen and filters.

Appendix: API Contract Reference

Full JSON examples and every error message are in `API_CONTRACT.md`. That file is the source of truth; this table is only a summary.

Route	Input	Success	Owner
GET /events	query: category, date (YYYY-MM-DD), availableOnly=true, includePast=true	200 + Event[]	Majed
GET /events/categories	none	200 + string[]	Majed

GET /events/:id	none	200 + Event	Majed
POST /bookings	body: eventId, customerName, customerPhone, places	201 + Booking	Adham
POST /bookings/find	body: phone, code	200 + Booking	Adham
POST /bookings/:id/cancel	body: phone	200 + Booking	Adham

Event: id, title, description, date (YYYY-MM-DD), time (HH:mm), location, category, capacity, booked, remaining, bookingPercentage, status

status: "Available" | "Almost full" | "Full" | "Past"

Booking: id, code (EV-XXXX), eventId, event (full Event, with updated availability), customerName, customerPhone, places, bookedAt, status ("Active" | "Cancelled"), canCancel (boolean)

Error shape (every failure): { statusCode, message }, where message is one readable sentence ready to show to the user. Codes: 400 invalid input, 404 not found, 409 rule conflict (full, not enough places, duplicate, past event, already cancelled, event started).

This contract is a proposal. The team leader confirms the final version; if a field name changes, the leader updates this reference.